

Pixel-DNA: Increasing Robustness of Approximate DNA Storage for Images by Using Hierarchical Deduplication

Alex Sensintaffar*, David Du[†], Bingzhe Li*

*The University of Texas at Dallas, Richardson, TX, USA

[†]University of Minnesota, Twin Cities, Minneapolis, MN, USA
{alex.sensintaffar, bingzhe.li}@utdallas.edu, du@umn.edu

Abstract—DNA is a high-density long-lasting storage medium that has the potential to keep up with the ever-increasing data storage needs. However, DNA storage is incredibly error-prone. The more nucleotides stored in the DNA strand the greater the chance of data loss. Moreover, DNA storage faces a specific phenomenon called error propagation due to its encoding from binary to nucleotide, resulting in an even higher error rate. When the majority of relevant data is stored inside the DNA strand error propagation is limited to the single strand. Reducing the overall data stored in DNA allows for more error correction and error protection. In this paper, we propose an isolation and deduplication scheme called Pixel-DNA for images. Pixel-DNA uses specific properties of DNA and DNA strands to implement multiple levels of deduplication while maximizing the data isolation. Also, a data segmentation scheme is applied to further increase the density of DNA storage. Based on the experimental results, Pixel-DNA allows for increased robustness and recoverability of images while increasing the density of DNA storage capacity.

Index Terms—DNA Storage, Deduplication, Images

I. INTRODUCTION

According to the International Data Corporation (IDC), global digital data is projected to reach 291 Zettabytes (ZB) by 2029 [1]. A significant portion of this data consists of images, which have seen explosive growth due to the widespread use of social media platforms like Facebook, Snapchat, and Instagram. Additionally, the need to maintain multiple versions of images at different resolutions for various devices and usage contexts further amplifies the demand for large-scale storage systems [2]. Consequently, it becomes essential for new storage technologies to accommodate vast amounts of image data efficiently and cost-effectively.

To address the ever-increasing demand for greater data density, researchers have worked to add improvements to current storage systems [3]–[5] such as Hard disk drives (HDD) and Solid state drives (SSD). These storage systems do provide an increase in storage density. However, the density increase of those devices cannot catch up with the rapidly increased rate of data generation. Moreover, there is an increasing need for long-term digital data storage. Those traditional storage devices have up to only about 30 years of lifetime at most.

*This work was partially supported by NSF 2204656, 2343863, 2413520 and 2440611.

Deoxyribonucleic Acid (DNA) storage has emerged as a promising storage medium for archiving data due to its property to offer both a high storage density and long-term preservation [6]–[9]. Theoretically, DNA storage has a density of about 455 Exabytes/gram [9], [10], and data stored with DNA can be preserved for more than hundreds of years [11], [12]. Consequently, DNA storage holds immense potential for accommodating vast amounts of data.

However, DNA storage is not without its limiting features. One important limiting feature is the error-prone aspect of DNA storage. DNA storage is incredibly susceptible to insertion, deletion, and substitution errors [13]. Due to limitations associated with the sequencing processes long strings of homopolymers and GC content violations can dramatically increase the overall error rate. Besides the errors introduced in the sequencing process, errors can occur during the synthesis and PCR process. The significant increase in errors associated with DNA storage when compared to other data storage mediums can result in an error-prone file being completely unusable for storage means. Moreover, due to the conversion between digital binary and nucleotide, and insertion and deletion errors, a special phenomenon called error propagation further increases the error rates in DNA storage [14], [15]. The error propagation indicates that one nucleotide error may cause a sequence of wrong reads on its following nucleotides. Thus, the error propagation further increases the error rates of the DNA storage system. Some previous studies focused on the mitigation of error propagation. For example, IMG-DNA [14] introduced barriers to prevent error propagation, however, decreasing the capacity of DNA storage. Lin et al. [15] proposed an unbalanced error correction code (ECC) to reduce the ECC overhead.

In this paper, we address the error propagation for images in DNA storage from a new perspective by reducing DNA strand length. We introduce Pixel-DNA, a four-phase scheme designed to limit errors within small pixel blocks while minimizing the number of nucleotides used to store images. First, images are divided into $N \times N$ pixel blocks, each stored in a separate strand, confining errors to individual blocks. Second, duplicate strands are replaced with index pointers, cutting nucleotide usage. Third, repeated index pointers are compacted into fewer strands. Fourth, we optimize strand

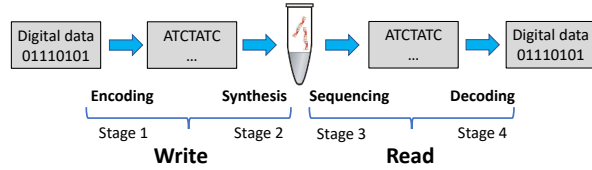


Fig. 1. DNA storage process.

size by chunking strands into per-pixel data, using a lookup table to replace frequent chunks with indices, further reducing nucleotide count.

The rest of this paper is organized as follows: Section II describes the background related to DNA storage and relevant information about images. Section III describes the motivation behind this work. Section IV provides an in-depth description of the four phases of the algorithm. Section V shows the experimental results comparing different schemes to store images in DNA. Section VI describes the conclusions drawn from the experimental results.

II. BACKGROUND

A. DNA Storage

Deoxyribonucleic acid (DNA) is a molecule that typically exists inside cells that contains the entire genetic information of an organism. DNA exists as two long polymers made from four repeating nucleotides. The four nucleotides are adenine (A), cytosine (C), thymine (T), and guanine (G). Adenine (A) and thymine (T) will bond together and cytosine (C) and guanine (G) will bond together. The long polymers are complementary to each other and therefore only one strand needs to be examined to obtain the stored information. Within the DNA storage process, there are four distinct stages as shown in Fig. 1. Stage 1 is encoding. Binary data is converted to DNA strands using an encoding scheme as described in Section II-B. Synthesis at Stage 2 virtually encodes DNA strands that will be written to a chemically synthesized DNA strand. Stage 3 is the sequencing process, which uses the Polymerase Chain Reaction (PCR) technology to amplify the target DNA strands. At Stage 3, the chemically synthesized DNA will be read into a virtual encoded DNA strand. Stage 4 is to decode the DNA strand and convert the nucleotides back to binary data.

B. DNA Encoding

DNA encoding and decoding convert binary data to DNA and vice versa [12], [16]–[20]. A simple binary-based method maps A, C $\rightarrow$ 1 and T, G $\rightarrow$ 0, yielding 1 bit per nucleotide. Homopolymers are avoided by alternating nucleotides, and GC balance is maintained if G/C represent different bits. A more common method is base 4 encoding: A $\rightarrow$ 00, C $\rightarrow$ 01, T $\rightarrow$ 10, G $\rightarrow$ 11, achieving 2 bits per nucleotide but not addressing homopolymers or GC content, while halving nucleotide usage. Some studies focus on bio-constraint-aware schemes. For instance, GF(47) [12] maps binary to Galois Field 47, then to nucleotide triplets. Rotation codes [21], [22]

use Huffman-based base-3 encoding, determining nucleotides based on prior values. Wei et al. [23] proposed a new mapping to increase DNA storage capacity and Brunmayr et al. [24] designed a motif generation tool to avoid bio-constraints. Our proposed Pixel-DNA supports any encoding scheme and by default we adopt base 4 as in [15].

C. DNA Synthesis and Sequencing

DNA Synthesis and Sequencing are the processes of writing (synthesis) and reading (sequencing) DNA strands as shown in Fig. 1. Typically DNA synthesis will create a DNA strand nucleotide by nucleotide [25]. The DNA synthesis process typically involves making hundreds of copies consisting of the same DNA sequence. Out of all the copies of the same DNA sequence, there will be multiple perfect strands [13], [22], [25]. The perfect strands are given a primer binder at the beginning and end of the DNA sequence along with an index after the beginning primer. The sequencing process can be separated into two parts. The first part PCR, is the process of doubling the number of a target DNA strand. The PCR process targets a DNA strand based on the assigned primers. The target DNA strand is first split into two "reverse complement" [22] DNA sequences. An enzyme known as polymerase will take a single-stranded DNA sequence and turn it into a double-stranded DNA strand. The result is two identical DNA strands. The PCR process is then repeated between 25 and 35 times to achieve a sufficient number of DNA strands [13], [22]. The second part of sequencing is the process of reading the nucleotides in a given DNA strand and converting them to a virtual DNA strand. While there are many ways to implement DNA sequencing one of the most popular ways is to take the target DNA strand and attach fluorescent nucleotides to each of the nucleotides on the DNA strand. The sequencing process can be applied to the DNA strand copies created during the PCR process. Given enough DNA strands an accurate virtual DNA strand can be obtained [22].

D. Image Background

Images are among the most widely stored file types, typically in three formats: 24-bit true color (raw), JPEG, and PNG. 24-bit true color images are uncompressed, with each pixel represented by 8-bit red, green, and blue values, allowing over 16 million colors. Although beyond human visual sensitivity, this format serves as a base for compression methods.

For the image compression schemes, JPEG (Joint Photographic Experts Group) is the most commonly seen image format. JPEG offers lossy data compression using discrete cosine transformation (DCT). JPEG compression starts by subdividing a grid into 8×8 pixel blocks. The DCT is then applied to the pixel blocks. By applying the DCT to the pixel block the pixel values are converted to the frequency domain. The low-frequency components of the pixel block offer much higher resolution while the high-frequency components have a negligible effect on the image quality. By only saving the low-frequency components of the pixel blocks significantly less data is needed to save the image.

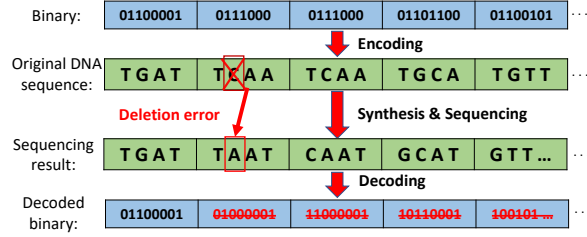


Fig. 2. An example of error propagation during the whole DNA storage process.

Even though the high-frequency components have a negligible effect on the image quality, the loss of the high-frequency components results in a lossy image compression scheme. PNG (Portable Network Graphics) is another lossless data compression format excellent at storing true color images and images with transparency. PNG uses a 2-stage compression process where the first stage uses a filtering prediction method to reduce the number of unique symbols. Each pixel is encoded based on the pixel's values above, to the left, and above-left of the pixel. The average of the three other pixels is calculated then the difference between the averages and the current pixel value is now stored. Not all three other pixels are used to calculate the average. The compression algorithm often uses the values that produce the least number of unique symbols. This now leads to the second stage of compression called deflate. Deflate primarily uses the Huffman encoding scheme to encode the symbols from the first stage. Huffman encoding uses a tree structure to assign all symbols given to a unique symbol. These unique symbols are shorter for more frequent given symbols and longer for fewer frequent symbols.

III. MOTIVATION

As discussed before, DNA storage is error-prone. There are three basic types of error, including insertion, deletion, and substitution. During synthesis and sequencing processes, nucleotides may not be successfully written to and read from DNA storage. After that, nucleotides need to be converted back to binary data, which may cause a special scenario named error propagation occurs in DNA storage [14], [15]. This scenario even increases the error rates in DNA storage. Fig. 2 provides an example of error propagation during the whole DNA storage process. When there is a deletion happens (i.e., C is deleted), its subsequent nucleotides will be shifted to one nucleotide left for the decoding purpose. As a result, the decoded binary data only has the first byte of data correct, and the rest of the data is totally wrong. Also, we apply an insertion error to a real image to graphically demonstrate the error propagation issue in DNA storage. As shown in Fig. 3, we insert one insertion error in an image. Due to the error propagation issue, only one single error causes significant image quality degradation. Only the first several rows of the image are fully recovered. The rest of the image is totally crashed.



(a) Original JPEG image



(b) The image with an insertion error at the 2444th nucleotide

Fig. 3. Graphical view of error propagation when injecting one insertion error to an image.

Although some previous studies focus on the error propagation issues by introducing new ECC codes or barriers, as biotechnology develops, the DNA strand length will increase their solution becomes less efficient. Due to the natural physical isolation between DNA strands, error propagation happens within the DNA strand if we have the proper metadata. Thus, if we can decrease the length of the original DNA strand, the error propagation can be mitigated. In this paper, we propose the hierarchical deduplication scheme to reduce the number of nucleotides so that the error propagation is mitigated. A detailed description is provided in the following sections.

IV. ALGORITHM DESIGN

In this section, we propose a DNA image hierarchical deduplication methodology that balances the capacity and robustness of the DNA storage system. It minimizes the effect of the error propagation on images while reducing the total number of nucleotides needed to store images.

A. Overall Architecture of Pixel-DNA

Pixel-DNA encoding consists of three main phases as shown in Fig. 4. The first phase is **Image Encoding Methodology** where the input images are converted to a pixel matrix. The subsequent pixel matrix is then converted into a list of DNA strands. The second phase is **First-Level Deduplication** where all of the DNA strands in the strand list are compared for duplicate strands. If a strand is found to be a duplicate of a

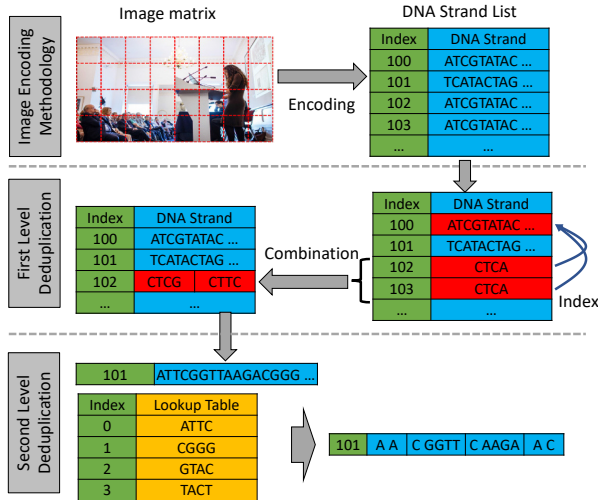


Fig. 4. The overall architecture of Pixel-DNA.

previous strand the duplicate strand is replaced with an index pointer strand. After that, duplicated pointers strands found in sequential order are combined into a single duplication block. The subsequent duplication blocks can be pulled from their respective location in the strand list and combined into a sub-strand list. The third phase is **Second-Level Deduplication** where each strand is divided into chunks of a fixed size. The most frequently seen chunks inside the strand list are added to a lookup table. Every instance of duplicate chunks found in the strand list is then replaced with the corresponding lookup table index.

B. Image Encoding Methodology

The first phase of Pixel-DNA is converting an image to a DNA strand list. The image is first converted to an RGB pixel matrix. The pixel matrix is separated into $N \times N$ blocks of pixels (Note that by default we use $N = 5$ in the paper to match the DNA strand length requirement in the DNA storage). Using an image segmentation process, the algorithm iterates through the pixel blocks both horizontally and vertically as shown in Fig. 6 and Fig. 7. The image segmentation direction that results in the fewest number of nucleotides will be used. The number of nucleotides between the image segmentation direction is based on compression from the First-Level Deduplication combination as described in Section IV-C. Once the direction is determined, each pixel block is converted to a single DNA strand. The image segmentation determines the ordering of the DNA strands within the DNA strand list. The strands are assigned an index in sequential order following the order of the list.

Each pixel inside the pixel block is converted into 12 nucleotides using the base 4 DNA encoding method described in Section II-B. Four nucleotides are used to represent the eight bits of a single RGB color. Four nucleotides per color, three colors per pixel, and 25 pixels per block result in a strand of

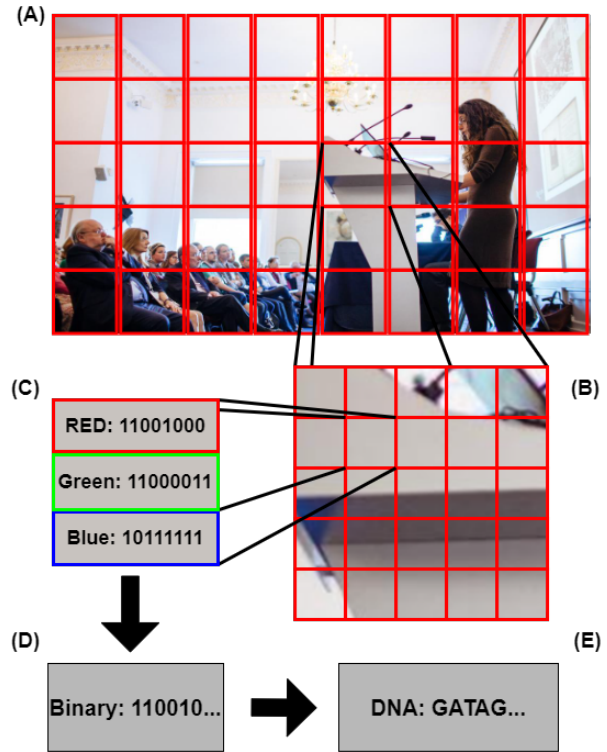


Fig. 5. Process of extracting DNA sequences from images. (A) The original image is separated into a grid of 5×5 pixel blocks. (B) Each 5×5 pixel block is separated from the image following the image segmentation path described in Section IV-B. (C) The 24-bit RGB values from each pixel inside the pixel block, are combined into a single binary string. (D) The binary strings from each pixel in the pixel block are combined into a single binary string 600 bits long. (E) Using the base 4 encoding scheme each pixel block binary string is converted to a DNA strand 300 nt long. The process results in one DNA strand per pixel block.

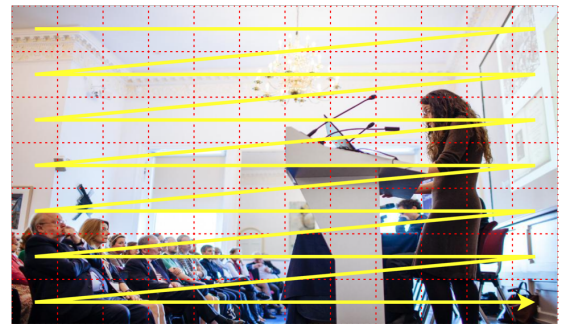


Fig. 6. Image segmentation following the horizontal path through the image.

300 nucleotides long. The result of this phase is a list of DNA strands with each strand 300 nucleotides long. Note that due to the bio-constraints, most of the existing works for DNA storage use 100–300 nt length of a DNA strand. In this study, we use 300 nt by default, and other DNA strand lengths can be applied to Pixel-DNA as well.

TABLE I
DUPLICATION BLOCK MULTIPLIER AND SIZE RELATIONS

$Size_i / Size_p$	Multiplier=0=A (1+0)(X)+1		Multiplier=1=C (1+1)(X)+1		Multiplier=2=T (1+2)(X)+1		Multiplier=3=G (1+3)(X)+1	
	Number of Nucleotides	Number of Permutations	Number of Nucleotides	Number of Permutations	Number of Nucleotides	Number of Permutations	Number of Nucleotides	Number of Permutations
A→00	1	4	1	4	1	4	1	4
C→01	2	16	3	64	4	256	5	1,024
T→10	3	64	5	1,024	7	16,384	9	262,144
G→11	4	256	7	16,384	10	1,048,576	13	16,777,216

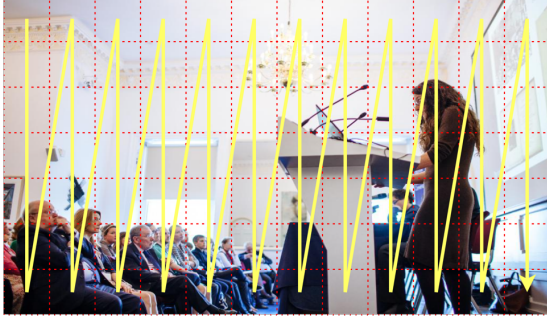


Fig. 7. Image segmentation following the vertical path through the image.

C. First-Level Deduplication

The First-Level Deduplication phase looks for strands inside the strand list that are duplicates and replaces them with a shorter DNA strand pointer. Pixel-DNA iterates through the strand list looking for duplicate strands. The first time Pixel-DNA sees a unique strand, the strand and index is stored. When Pixel-DNA sees a duplicate strand, the strand is replaced with a DNA representation of the first instance index. The overall number of nucleotides used to represent the image is reduced. By using the pixel blocks described in Section IV-B DNA strands will be more uniform in the pixel colors being represented. Pixel blocks next to each other on the image are more likely to create the same strand thus increasing the hit rate of first-level deduplication.

Another way to increase the hit rate of the first-level deduplication scheme is to introduce an acceptable difference. The acceptable difference is a set number of nucleotides between two strands that can be different and still allow for the strand to be considered a duplicate. The acceptable difference can increase the total number of first-level deduplication hit in a given strand list. The acceptable difference will make the first-Level Deduplication phase lossy but the given loss will be negligible compared to the loss introduced in storing DNA.

After indexing the duplicated DNA strands, the resulting strand list consists of many short pointer strands. Often times the same pointer strands are repeated for many indexes in a row. These strands can be combined into a single strand called a deduplication block. To properly recover the combined strands the *start index*, *stop index*, and *strand pointer* must be properly saved. The initial scheme design had all three data

points represented using the same number of nucleotides based on whichever data point needed the most nucleotides. This resulted in a large number of excess nucleotides added to the data points to keep the all the same size.

To further reduce the number of nucleotides a dynamic scheme is introduced that has the *StartIndex* and *StopIndex* the same number of nucleotides and the *StrandPointer* a different number of nucleotide based on their respective minimum needs. To accomplish this three new data points are introduced; *multiplier*, *size_i*, and *size_p*. Each of the new data points is a single nucleotide long with each of the possible nucleotide values representing an integer using the base 4 encoding. To derive the new data point values first the minimum length of the stop index and strand pointer is calculated using (1) and (2). To ensure the longer data point can be represented (3) is used to determine what data point is calculated first. The multiplier is then calculated based on the value of λ from (3) using (4). Each of the different values for tempSize is tested in sequential order until an acceptable value for the multiplier is found in the range of [0, 3]. Using the calculated values from (3) and (4) the first size data point is found by using (5). If λ_i is greater than $size_i$ else $size_p$ is calculated first. Once the first size data point is calculated then the size data point not yet calculated is found using (6). Just like with 4 the newSize is calculated using values in the range of [0,3] until newSize is a value within the range [0,3]. Table I shows the number of nucleotides the different values of *Size_i* and *Size_p* can represent based on a given multiplier.

$$\lambda_i = \text{ceil}(\log_4(\text{StopIndex})) \quad (1)$$

$$\lambda_p = \text{ceil}(\log_4(\text{StrandPointer})) \quad (2)$$

$$\lambda = \max\{\lambda_i, \lambda_p\} \quad (3)$$

$$\text{multiplier} = \frac{\lambda - 1}{\text{tempSize}} - 1; \text{tempSize} = 0, 1, 2, 3 \quad (4)$$

$$\text{size} = \frac{\lambda - 1}{\text{multiplier} + 1} \quad (5)$$

$$\text{newSize} = (\text{multiplier} + 1) \cdot \text{tmp} + 1; \text{tmp} = 0, 1, 2, 3 \quad (6)$$

Each duplicate block contains the start index, stop index, and data pointer thus allowing for the duplicate blocks to be pulled out of the strand list. The duplicate blocks can be combined into a grouping of strands known as the combination group. The combination group is then placed at the beginning

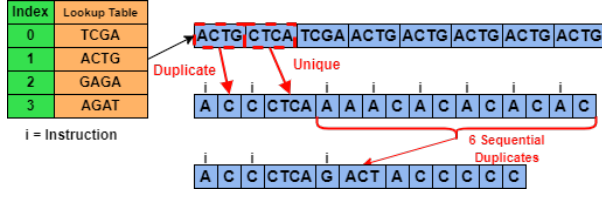


Fig. 8. Second-Level Deduplication encoding a duplicate chunk, unique chunk, and multiple sequential duplicate chunks

of the strand list. Because the combination group represents a significantly large amount of data in a few strands future work can add an excess number of error correction codes to this group while creating a small amount of excess data creation.

The combination at First-Level Deduplication can be decoded by using (7) and (8).

$$\text{Len}(\text{StopIndex}) = (1 + \text{multiplier}) \cdot (\text{size}_i) + 1 \quad (7)$$

$$\text{Len}(\text{StrandPointer}) = (1 + \text{multiplier}) \cdot (\text{size}_p) + 1 \quad (8)$$

D. Second-Level Deduplication

Second-level deduplication reduces the total number of nucleotides needed to represent an image by replacing duplicate chunks inside a DNA strand with a lookup table index. All DNA strands not affected by First-Level Deduplication are divided into chunks 12 nt long. The most frequently seen chunks will be added to a lookup table. All chunks are then labeled as duplicates (chunks that exist in the lookup table) or unique (chunks not stored in the lookup table). To increase the total number of duplicate chunks an acceptable difference between the chunks and lookup table values can be implemented. Once all chunks have been labeled as unique and duplicate the chunks are replaced with a *recipe chunk*. A recipe chunk consists of one nucleotide instruction and the relevant data.

Instruction	Chunk Type
A	Duplicate Chunk
C	Unique Chunk
T	Long Sequence of Duplicate Chunks
G	Long Sequence of Unique Chunks

There are four different types of recipe chunks. 1) A duplicate chunk is replaced with the lookup table index for the given duplicate value and given the instruction A. 2) Unique chunks are simply given the instruction value C and the data is kept the same. 3) In the event of multiple duplicate chunks occurring with no unique chunks between, the duplicate chunks can be combined into a single recipe block. This recipe block starts with the instruction T and then a set of 3 nt to indicate the number of duplicate chunks included in the recipe chunk. The rest of the recipe chunk is the lookup table index values in respective order. 4) In the event of multiple unique chunks occurring with no duplicate chunks between, the unique chunks can be combined into a single

recipe block. This recipe block starts with the instruction G followed by a set of 3 nt to indicate the number of unique chunks included in the recipe chunk. The rest of the recipe chunk is the original chunk values in respective order.

V. EXPERIMENTAL RESULTS

A. Environmental Setup

System setup: Experimental results are obtained using Python 3.8.10 to encode images into DNA sequences based on different schemes. The system is used to run the experiments with an Intel Xeon Silver 4214 CPU@2.2 GHz with 32 GB memory.

DNA configurations: DNA strands are encoded using a base 4 encoding scheme as used in [15] where two bits correspond to one nucleotide. In this instance 00→A, 01→C, 10→T, 11→G. All DNA strands are initially encoded to be 300 nt long but have the potential to change length after Pixel-DNA is applied to the image.

Dataset: Image dataset uses the Open Images Challenge 2019 detection dataset [26]. This dataset provided almost 10,000 JPEG images with varying pixel dimensions ranging between 270 and 1024 pixels. To evaluate large-scale images, PNG images from [27], which provides ultra-high definition images at 2K, 4K, and 8K resolution are also used. Fig. 9 and Fig. 10 do not include results from [27] to keep the number of nucleotides in relation to JPEG images. Overall, the test dataset includes approximately 108,000 images with 36.1 GB. Images are randomly pulled from the test dataset.

Metrics: To evaluate the robustness of DNA storage, a metric called SSIM (structural similarity index metric) [28] is used to indicate the quality of images. The SSIM value is in the range [-1, 1]. The larger SSIM value indicates that the two images are close to each other. So, if two images are near-identical, the SSIM will be close to 1. Another metric is named peak signal-to-noise ratio (PSNR) to evaluate the similarity between original and recovered images [29]. For the capacity of the DNA storage system, we use the most straightforward way - the number of nucleotides, which indicates how many nucleotides are needed to store an image dataset. Fewer nucleotides used mean higher DNA storage capacity.

Baselines: We compare our proposed Pixel-DNA with three other baselines: the scheme in [15] based on raw image data (noted with Raw), JPEG/PNG data (noted by Raw-JPG) and our scheme on top of JPEG/PNG (noted by Pixel-JPG).

B. Overall Comparison

In this section, we investigate the comparison between the capacity and robustness of different schemes by injecting different error rates. The error rate injection follows the error distribution from a wet-lab result [21]. The error rates are injected from 0% to 2%.

As shown in Fig. 9 and Fig. 10, from the capacity perspective, JPEG-based schemes (i.e., Raw-JPG and Pixel-JPG) achieve the least number of nucleotides since the JPEG scheme

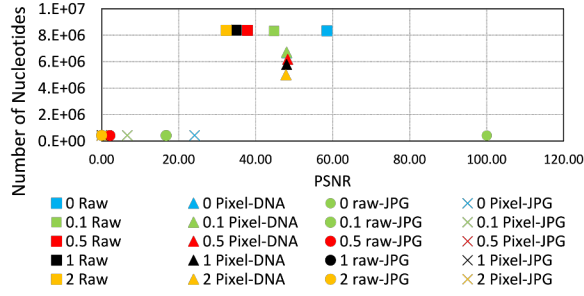


Fig. 9. Capacity vs. PSNR with injecting different error rates. With the legend format of X% Y, X refers to the error rate and Y refers to the scheme.

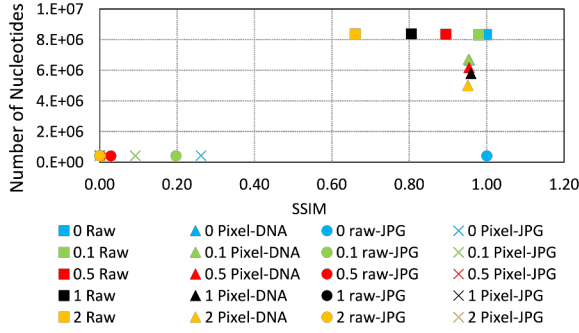


Fig. 10. Capacity vs. SSIM with injecting different error rates. With the legend format of X% Y, X refers to the error rate and Y refers to the scheme.

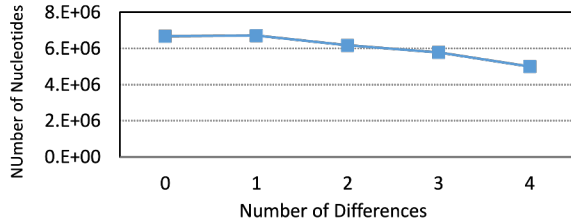


Fig. 11. Number of nucleotides as the number of differences increases in Pixel-DNA.

can achieve a much higher compression ratio. However, JPEG-based schemes obtain really bad fault tolerance. When injecting a small error rate, the PSNR and SSIM values of those two JPEG-based schemes perform extremely poorly compared to the Raw and Pixel-DNA schemes. For Pixel-DNA, as increasing the error rate, there is little impact on the robustness. In other words, SSIM and PSNR values remain similar with different injection errors for Pixel-DNA, and Pixel-DNA obtains strong robustness. Also, as the error rate increases, the capacity of Pixel-DNA is increased. This is because the errors may generate duplicate DNA nucleotide patterns and further improve the deduplication ratio for Pixel-DNA. Another interesting observation is that when applying our Pixel-DNA to JPEG/PNG schemes, the capacity can achieve a slight improvement but the robustness remains the

same. This is because the compression schemes in JPEG/PNG have a strong dependency between nucleotides, which is significantly sensitive to error propagation in DNA storage. So, this is the reason why we do not use Pixel-DNA on top of image compression schemes. Fig. 12 provides a graphical view of different schemes with injecting 0.1% error. Raw and Pixel-DNA obtain the best image quality compared to others. In summary, Pixel-DNA achieves the best balance between the capacity and robustness of DNA storage compared to the other schemes.

C. Varying the Number of Differences

During the proposed hierarchical deduplication, we allow the difference between the referenced DNA pattern and the target DNA patterns. The more differences the scheme uses, the higher the DNA capacity the scheme has. In this subsection, we investigate the influence of the number of differences in deduplication on capacity and robustness.

As shown in Fig. 11, it follows the intuition that as the number of differences increases in the deduplication process, the number of nucleotides in DNA storage is decreased. The capacity can be raised by about 34% when the number of differences increases from 0 to 4. For robustness, as the number of differences increases, both PSNR and SSIM values are slightly decreased. The major reason for the reduction of image quality is that as introducing more differences, to recover the DNA strands from deduplication, some errors will be introduced. However, the DNA strand length will be decreased due to a higher deduplication ratio, which can increase the robustness of the error propagation. As a result, these two factors fight with each other and cause a slight decrease in robustness in Pixel-DNA.

D. Breakdown Analysis

In this subsection, we discuss the effect of different techniques on the capacity and robustness of DNA storage. Starting from the Raw scheme [15] as a baseline, we add one feature each time until Pixel-DNA is obtained as described below:

- **Raw:** is the baseline scheme.
- **FLD:** only uses First-Level Deduplication (FLD) on top of the raw scheme.
- **SLD:** applies Second-Level Deduplication (SLD) on top of the raw scheme.
- **Pixel-DNA:** is our proposed Pixel-DNA scheme with both FLD and SLD.

Fig. 14 (a) provides the breakdown analysis for Pixel-DNA. For the capacity, from Raw to Pixel-DNA, each technique can contribute some capacity improvement. Among them, Second-Level Deduplication provides the most significant improvement compared to others. The reason for that is Second-Level Deduplication can reduce the number of nucleotides in every strand. First-Level Deduplication can reduce the total number of nucleotides in a duplicate strand significantly more than Second-Level Deduplication but has no effect on unique strands. For most images, the total number of unique strands greatly outnumbers the number of duplicate strands

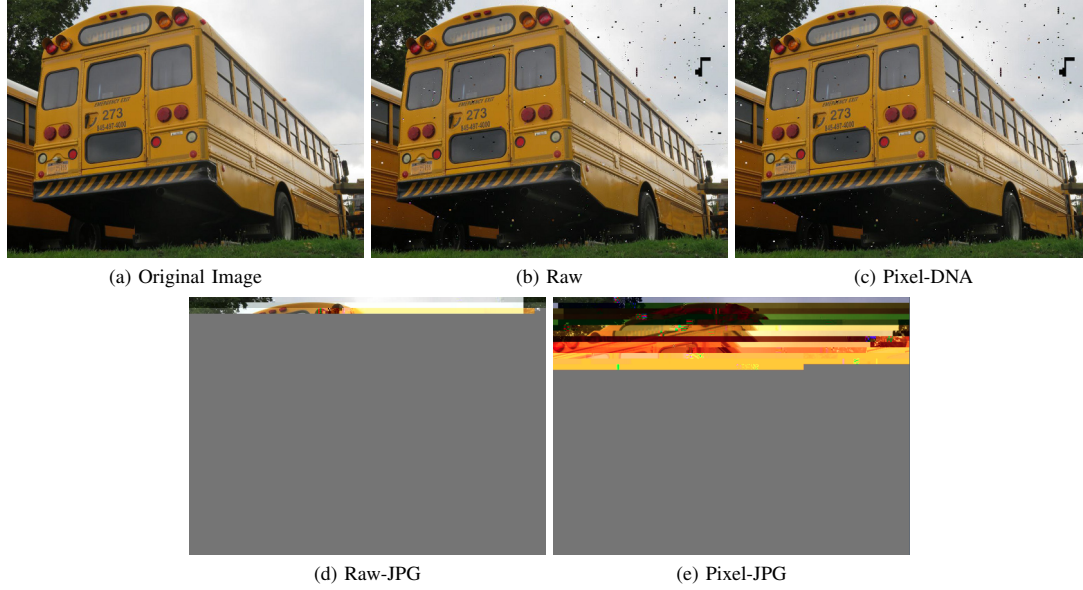


Fig. 12. All images (besides the original image) have a 0.1% injection error rate

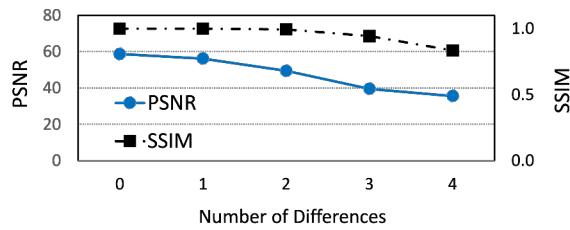
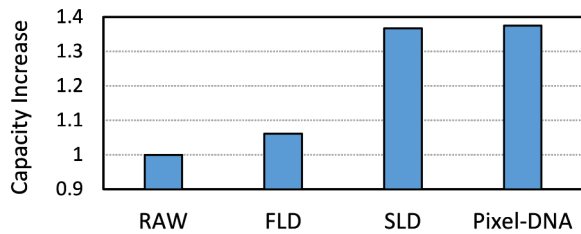
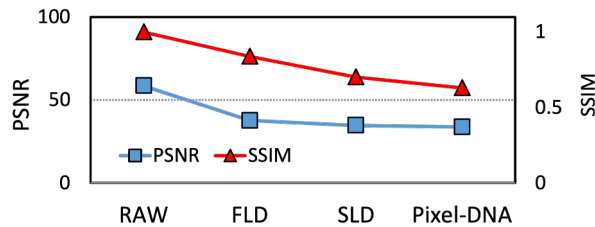


Fig. 13. PSNR and SSIM values as the number of differences increases in Pixel-DNA.



(a) Capacity



(b) Robustness

Fig. 14. Breakdown analysis for Pixel-DNA.

thus Second-Level Deduplication reduces the total number of nucleotides by reducing the number of nucleotides in all strands. For errors, according to Fig. 14 (b), we can find that the First-Level Deduplication scheme provides the most robustness to the DNA storage system. When either First-Level Deduplication or Second-Level Deduplication incurs any insertion or deletion error the entire strand and the subsequent pixel block could become corrupt. When First-Level deduplication encounters a substitution error the worst case is a single pixel becomes corrupt. Second-Level Deduplication is still vulnerable to substitution errors because it uses specific nucleotides as instructions or as length indicators. Should any instruction or length indicator be changed the entire strand could be completely lost.

VI. CONCLUSION

In this paper, we propose an isolation and deduplication scheme called Pixel-DNA for images. Pixel-DNA uses specific properties of DNA and DNA strands to implement multiple levels of deduplication while maximizing the data isolation. Also, a data segmentation scheme is applied to further increase the density of DNA storage. Based on the experimental results, Pixel-DNA shows a clear decrease in the number of nucleotides and an improvement on robustness. Future work associated with Pixel-DNA would be to perform better tests. More images need to be tested to create the experimental results. Images of varying sizes need to also be tested. A robustness test using the PSNR and SSIM metrics needs to be implemented on the test data. The test data also need to have varying parameters to find better results. The lossy data implementation needs to be implemented and tested using various acceptable difference parameters.

VII. ACKNOWLEDGMENT

This work was partially supported by NSF 2204656, 2343863, 2413520 and 2440611. Any opinions, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

REFERENCES

- [1] "Idc worldwide global datasphere forecast, 2023-2027," <https://www.idc.com/getdoc.jsp?containerId=US50554523>.
- [2] E. Yan, K. Zhang, X. Wang, K. Strauss, and L. Ceze, "Customizing progressive jpeg for efficient image storage," in *9th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 17)*, 2017.
- [3] M. H. Hajkazemi, A. N. Kulkarni, P. Desnoyers, and T. R. Feldman, "Track-based translation layers for interlaced magnetic recording," in *USENIX Annual Technical Conference*, 2019, pp. 821–832.
- [4] H. Chen, Y. Lv, C. Li, S. Gu, and L. Shi, "An empirical study of hybrid ssd with optane and qlc flash," in *2020 IEEE 38th International Conference on Computer Design (ICCD)*. IEEE, 2020, pp. 175–178.
- [5] B. Li, C. Deng, J. Yang, D. Lilja, B. Yuan, and D. Du, "Haml-ssd: A hardware accelerated hotness-aware machine learning based ssd management," in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2019, pp. 1–8.
- [6] Y. Dong, F. Sun, Z. Ping, Q. Ouyang, and L. Qian, "Dna storage: research landscape and future prospects," *National Science Review*, vol. 7, no. 6, pp. 1092–1107, 2020.
- [7] A. Sensintaffar, Y. Wei, L. Ou, D. Du, and B. Li, "Advancing archival data storage: The promises and challenges of dna storage system," *ACM Transactions on Storage*, 2025.
- [8] Y. Hao, Q. Li, C. Fan, and F. Wang, "Data storage based on dna," *Small Structures*, vol. 2, no. 2, p. 2000046, 2021.
- [9] B. Li, N. Y. Song, L. Ou, and D. H. Du, "Can we store the whole world's data in {DNA} storage?" in *12th USENIX workshop on hot topics in storage and file systems (HotStorage 20)*, 2020.
- [10] R. Appuswamy, K. Lebrigand, P. Barbry, M. Antonini, O. Madderson, P. Freemont, J. MacDonald, and T. Heinis, "Oligoarchive: Using dna in the dbms storage hierarchy," in *Biennial Conference on Innovative Data Systems Research (CIDR 2019)*, 2019, p. p98.
- [11] M. E. Allentoft, M. Collins, D. Harker, J. Haile, C. L. Oskam, M. L. Hale, P. F. Campos, J. A. Samaniego, M. T. P. Gilbert, E. Willerslev *et al.*, "The half-life of dna in bone: measuring decay kinetics in 158 dated fossils," *Proceedings of the Royal Society B: Biological Sciences*, vol. 279, no. 1748, pp. 4724–4733, 2012.
- [12] R. N. Grass, R. Heckel, M. Puddu, D. Paunescu, and W. J. Stark, "Robust chemical preservation of digital information on dna in silica with error-correcting codes," *Angewandte Chemie International Edition*, vol. 54, no. 8, pp. 2552–2555, 2015.
- [13] Y. Wei, B. Li, and D. H. Du, "Dna storage: A promising large scale archival storage?" *arXiv preprint arXiv:2204.01870*, 2022.
- [14] B. Li, L. Ou, and D. Du, "Img-dna: approximate dna storage for images," in *Proceedings of the 14th ACM International Conference on Systems and Storage*, 2021, pp. 1–9.
- [15] D. Lin, Y. Tabatabaee, Y. Pote, and D. Jevdjic, "Managing reliability skew in dna storage," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 482–494.
- [16] L. Anavy, I. Vaknin, O. Atar, R. Amit, and Z. Yakhini, "Improved dna based storage capacity and fidelity using composite dna letters," *bioRxiv*, p. 433524, 2018.
- [17] G. M. Church, Y. Gao, and S. Kosuri, "Next-generation digital information storage in dna," *Science*, vol. 337, no. 6102, pp. 1628–1628, 2012.
- [18] Y. Erlich and D. Zielinski, "Dna fountain enables a robust and efficient storage architecture," *science*, vol. 355, no. 6328, pp. 950–954, 2017.
- [19] N. Goldman, P. Bertone, S. Chen, C. Dessimoz, E. M. LeProust, B. Sipos, and E. Birney, "Towards practical, high-capacity, low-maintenance information storage in synthesized dna," *nature*, vol. 494, no. 7435, pp. 77–80, 2013.
- [20] Y. Li, D. H. Du, L. Ou, and B. Li, "Hl-dna: a hybrid lossy/lossless encoding scheme to enhance dna storage density and robustness for images," in *2022 IEEE 40th International Conference on Computer Design (ICCD)*. IEEE, 2022, pp. 434–442.
- [21] L. Organick, S. D. Ang, Y.-J. Chen, R. Lopez, S. Yekhanin, K. Makarychev, M. Z. Racz, G. Kamath, P. Gopalan, B. Nguyen *et al.*, "Random access in large-scale dna data storage," *Nature biotechnology*, vol. 36, no. 3, pp. 242–248, 2018.
- [22] J. Bornholt, R. Lopez, D. M. Carmean, L. Ceze, G. Seelig, and K. Strauss, "A dna-based archival storage system," in *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*, 2016, pp. 637–649.
- [23] Y. Wei, B. Li, and D. H. Du, "An encoding scheme to enlarge practical dna storage capacity by reducing primer-payload collisions," in *Proceedings of the Twenty-Nine International Conference on Architectural Support for Programming Languages and Operating Systems*, 2024.
- [24] S. Brunmayr, O. S. Sella, and T. Heinis, "Dna data storage: A generative tool for motif-based dna storage," in *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, 2025, pp. 573–581.
- [25] S. Kosuri and G. M. Church, "Large-scale de novo dna synthesis: technologies and applications," *Nature methods*, vol. 11, no. 5, pp. 499–507, 2014.
- [26] "Open images challenge 2019 detection," <https://storage.googleapis.com/openimages/web/challenge2019.html>, accessed: May 20, 2023.
- [27] K. Zhang, D. Li, W. Luo, W. Ren, B. Stenger, W. Liu, H. Li, and M.-H. Yang, "Benchmarking ultra-high-definition image super-resolution," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 14769–14778.
- [28] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [29] Y.-C. Kuo, R.-F. Chiu, and R.-S. Liu, "Long-term jpeg data protection and recovery for nand flash-based solid-state storage," in *2019 35th Symposium on Mass Storage Systems and Technologies (MSST)*. IEEE, 2019, pp. 141–147.
- [30] D. Dua and C. Graff, "UCI machine learning repository," 2017. [Online]. Available: <http://archive.ics.uci.edu/ml>
- [31] N. Asuni and A. Giachetti, "Testimages: a large-scale archive for testing visual devices and basic image processing algorithms," in *STAG*, 2014, pp. 63–70.
- [32] —, "Testimages: A large data archive for display and algorithm testing," *Journal of Graphics Tools*, vol. 17, no. 4, pp. 113–125, 2013.
- [33] J. J. Hunt, K.-P. Vo, and W. F. Tichy, "Delta algorithms: An empirical analysis," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 7, no. 2, pp. 192–214, 1998.
- [34] Y. Sun, G. Han, C. Liu, Y. Wang, and Y. Fang, "An asymmetric-error-aware ldpc decoding algorithm for dna storage," *IEEE Communications Letters*, 2022.
- [35] "Python: Peak signal-to-noise ratio (psnr)," Feb 2020. [Online]. Available: <https://www.geeksforgeeks.org/python-peak-signal-to-noise-ratio-psnr/>
- [36] R. R. Garafutdinov, D. A. Chemeris, A. R. Sakhabutdinova, O. Y. Kiryanova, C. I. Mikhaylenko, and A. V. Chemeris, "Encoding of non-biological information for its long-term storage in dna," *Biosystems*, p. 104664, 2022.
- [37] "TensorFlow Datasets, a collection of ready-to-use datasets," <https://www.tensorflow.org/datasets>.